

PROO/SEW III - Assignment: Movie Ratings

Objective

Create an application for managing movie ratings using *JavaFX* and *JDBC* in a table.

Things To Learn

- Using the *Model-View-Controller*-pattern with the model following the *Singleton*-pattern.
- Applying *Service Layer* and *Repository*-patterns to separate business logic and data access.
- Binding a `TableView` to an *observable list*.
- Wrapping lists to make them *read-only*, *filterable* and *sortable*.
- Applying *Java's Stream API*.
- Using *JDBC*.

Submission Guidelines

- Your implemented solution as **zipped** *IntelliJ*-project.

Task

For this assignment your goal is the implementation of a simple movie rating application using the *MVC*-pattern with separate *Service*- and *Repository*-layers, as well as a database for data persistence.

Model

Each movie rating consists of a *non-empty* title, a year above 1888 (apparently the year of the first movie) and a rating between 1.0 and 5.0 - throw an `IllegalArgumentException` otherwise! Two movies are *equal*, if they have the same title *and* year.

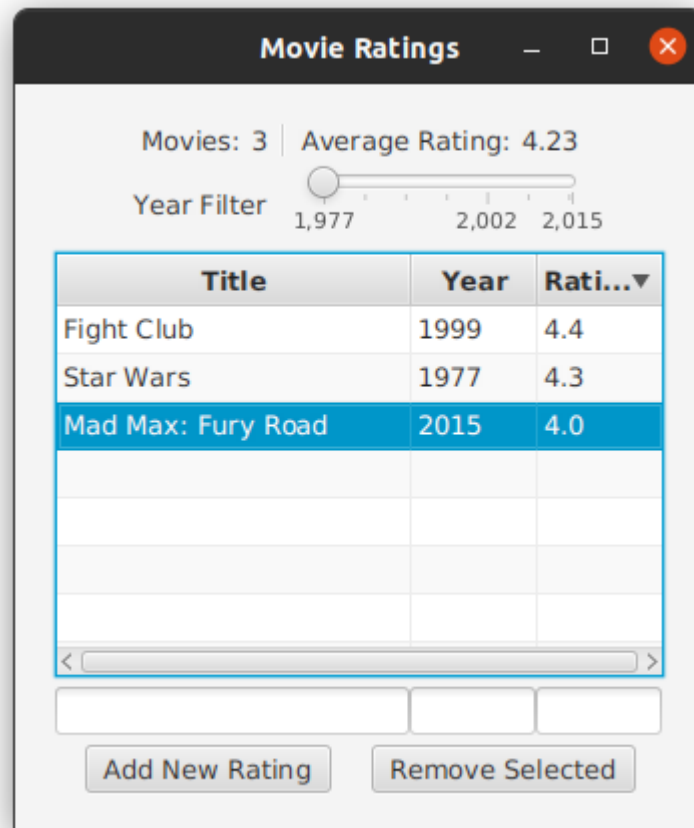
Service and Repository

Implement the *business logic* and the *data access* in their respective layers:

- Both the *Service* and the *Repository* should follow the *Singleton design pattern* using a *lazy initialization* approach.
- Store the ratings inside the `RatingsService` using an `ObservableList`.
 - Expose the list through a getter as an *unmodifiable* list.
 - Use `Simple???Property` fields for statistics that should be bindable to UI controls (e.g. the slider used for filtering).
- Implement methods for adding and removing movie ratings in the database inside your `RatingsRepository`.

- Use `PreparedStatement` and ensure all resources are closed properly using *try-with-resources*.
- Throw an exception if a movie rating that is being added already exists.

View And Controller



Recreate the *GUI* seen above inside your `.fxml`-file. Afterwards, implement the *controller* while considering the following:

- Show an **Alert** when you receive an exception from the model - e.g. when adding new ratings.
- Don't forget to replace the generic types of the `TableView` and the `TableColumns` - e.g. from `TableColumn<?, ?>` `tbcTitle` to `TableColumn<MovieRating, String>` `tbcTitle`.
- Wrap the list you receive from the model inside a `FilteredList` inside a `SortedList` - that way you can both apply filters and sort it by clicking on the column headers. Don't forget the following command:

```
moviesSorted.comparatorProperty().bind(this.tbvMovies.comparatorProperty());
```

- Bind both the labels at the top displaying the movie count and the average rating, as well as the *slider's* `min-` and `maxProperty` to the *properties* from the model - that way you don't have to worry about missing any changes!
- You can react to changes of the *slider* by adding a *listener* to its `valueProperty`.
 - Make sure to wrap the code of the anonymous function inside a `Platform.runLater()`-block to avoid concurrency issues!